

Проектное описание по программе Продвинутая разработка и тестирование ИИ-агентов

Как получить зачет

Для зачета по программе нужно выполнить **минимум одно из трех основных заданий на выбор**:

1. Каждое из трех заданий самостоятельно и **достаточно для зачета**.
2. Хотите сделать больше – можно сделать несколько заданий.
3. Отдельно есть **дополнительное** задание по оценке ИИ-агентов (оценка строится поверх готового агента) – вы также можете его выполнить.

Общие правила

- Модель: **GigaChat**
- Проект оформляется как **отдельная папка с README** и воспроизводимым запуском. Код понятный и читаемый, нетривиальные функции и классы снабжены docstring.
- Есть **скринкаст** (запись экрана с комментариями автора, до 10 минут) с демонстрацией работы вашего решения.

Где сдавать. Работа с репозиторием

Перейдите по [ссылке \(https://git.standard-data.ru\)](https://git.standard-data.ru) и авторизуйтесь под своими учётными данными (**высланные куратором логин и пароль**). В разделе групп <https://git.standard-data.ru/dashboard/groups> вас уже ждет ваша папка для сдачи проекта!

НЕ создавайте папку проекта самостоятельно! Преподаватели не увидят ее и не смогут проверить ваш проект.

Дедлайны и ключевые даты

24 - 26 августа	Проведение интенсива
1 сентября (18.00)	Консультация по проекту
4 сентября (18.00)	Консультация по проекту
<u>6 сентября</u>	<u>Срок сдачи проекта</u>

Сроки проверки проектов

Срок проверки присланных проектов – **3 дня**.

Ниже три основных задания, а также дополнительное задание по оценке ИИ-агентов. Для зачета по программе нужно выполнить хотя бы одно задание.

Задание 1. Ассистент с памятью

Необходимо подготовить индивидуальный проект: ассистент с долговременной памятью на GigaChat и набор кейсов, который доказывает, что эта память работает.

С памятью есть неприятная особенность: понять, стало лучше или нет, на глаз невозможно. Агент отвечает связно и когда помнит, и когда выдумывает, а разница всплывает через неделю в проде. Поэтому сначала вы собираете память, а потом собираете набор кейсов и проверяете им, работает она или только кажется. Проект должен:

- вести диалог с клиентом, сохраняя опыт между сессиями: извлекать факты, писать эпизоды, обновлять профиль;
- размечать записи по типам, назначать им срок жизни и читать их с порогом, посчитанным по своим данным;
- иметь собственный набор кейсов, проверяющий именно память: помнит ли агент сказанное в прошлой сессии, не подставляет ли лишнее, не выдаёт ли устаревшее;
- иметь размеченную часть набора, посчитанное согласие двух разметок и разбор того, где они разошлись;
- показывать, что изменилось после починки: те же кейсы на первой версии памяти и на исправленной, с цифрами и стоимостью.

Обязательные критерии

Выполненное задание должно удовлетворять 6 обязательным критериям.

№	Критерий	Занятие
1	Собран ассистент с внешней памятью на Mem0 поверх GigaChat: точки записи и чтения расставлены явно в цикле агента, записи размечены по типам (семантическая, эпизодическая, процедурная).	5

№	Критерий	Занятие
2	Память настроена осознанно, а не по умолчанию: записям назначен срок жизни, порог чтения посчитан по распределению близостей на своих данных и проверен заведомо посторонним запросом, промпт собран так, что кэш контекста попадает, экономия подтверждена числом.	5
3	Собран первичный набор кейсов на память: у каждого кейса есть состояние мира (что агент знал до), проверяемый эталон ответа и эталонная траектория в виде ограничений (must_call, must_not_call, порядок, потолок шагов); посчитано покрытие сценариев.	10
4	Набор расширен параметризацией и синтезом модели; проверено покрытие признаков (а не только непохожесть формулировок), сняты дубли, зафиксирован баланс по классам и сложности.	10
5	Не меньше 20 кейсов размечены дважды: сначала по своей инструкции, потом заново через сутки, вслепую и в перемешанном порядке. Посчитано согласие между двумя разметками (каппа с квадратичными весами), разобрано, где и почему вы разошлись сами с собой, инструкция исправлена, спорные кейсы переразмечены. Отчёт приложен.	10
6	Один и тот же набор прогнан дважды: на первой версии памяти с настройками по умолчанию и на исправленной. Приложены цифры по качеству памяти и стоимости диалога и сказано, какой разнице в этих цифрах можно верить при вашем размере набора.	5–10

Метрики качества памяти

Считаете по своему набору. Как именно считать, решаете сами, но напишите это в отчете, иначе цифры нельзя будет понять. От чего оттолкнуться:

- доля вспомненного: в скольких кейсах агент использовал факт из прошлой сессии там, где должен был;
- доля лишнего: какая часть подставленных в промпт записей не понадобилась для ответа;
- доля устаревшего: как часто агент опирался на запись, срок которой истек;
- стоимость диалога: токены на запись памяти и на подстановку.

Что можно сделать сверх обязательного

Не требуется, но сильно украшает работу:

- сравнить свой слой памяти на 50 строк с Mem0 по качеству и стоимости на одном и том же наборе;

- заменить хранение памяти на раскладку по видам в духе Mirix и показать, что изменилось;
- собрать агента на deepagents с файловой системой как рабочей памятью и проверить тем же набором;
- добавить процедурную память: извлечь способ решения из удачных прогонов и показать сокращение числа шагов.

Данные для проекта

Выдается набор многосессионных диалогов с ассистентом поддержки: в ранних сессиях клиент называет факты о себе и своей задаче, в поздних ссылается на них, не повторяя. Эталонов в данных нет: их вы строите сами, это часть работы по занятию 10.

Как размечать, когда вы один

Проект делается в одиночку, а согласие считается между двумя разметками. Второй разметкой становится ваша же, сделанная заново через сутки. Так делают и в настоящих проектах, когда второго разметчика взять негде. Чтобы она не превратилась в самообман:

- между проходами проходит хотя бы день;
- во второй раз порядок кейсов перемешан;
- первые оценки не открываются, пока не проставлены все вторые;
- размечаете по написанной инструкции, а не по памяти о том, что ставили в прошлый раз.

Расхождения с самим собой это не про невнимательность. Это точный указатель на места, где инструкция допускает два прочтения. Их и правите.

Формат сдачи

- Отдельная папка со всеми необходимыми скриптами или колабом.
- Отчёт (файл в репозитории): схема кейса, отчёт о согласии двух разметок, таблица с цифрами до и после починки памяти, вывод о том, чему в этих числах можно верить.
- Код понятный и читаемый, функции и классы снабжены docstring.
- скринкаст до 10 минут с демонстрацией работы вашего решения.

Задание 2. Диалоговый агент с памятью и гибридным поиском

Необходимо подготовить индивидуальный проект: диалогового агента с памятью и гибридным поиском, построенного на GigaChat и LangGraph и работающего на вашей собственной предметной области.

Сценарий использования: пользователь ведёт с агентом длинный диалог, в котором не сообщает все нужные сведения сразу, а выдаёт их по частям, перебивает сам себя посторонними вопросами и возвращается к первоначальной теме через десяток реплик. Агент обязан собрать из этого диалога структурированный набор реквизитов, найти в вашей базе знаний документы, релевантные вопросу, ответить строго со ссылкой на найденный источник и при этом накопить о собеседнике долговременные сведения, которые переживут завершение текущей сессии и будут корректно использованы при следующем обращении. Ключевая трудность, вокруг которой построен проект: контекстное окно модели конечно, база знаний со временем меняется, а сведения о пользователе приходят из источников разной надёжности и регулярно противоречат друг другу. Функционал проекта:

- Агент сохраняет контекст диалога в течение минимум 30 реплик с пользователем, а также имеет доступ к данным пользователя из предыдущих сессий.
- Поиск по базе знаний находит документы и по смыслу, и по точным идентификаторам, а также отбирает именно ту редакцию документа, которая была актуальна на нужный момент времени.
- Память о пользователе спроектирована как система с явной схемой записи, заданным горизонтом хранения, правилами разрешения противоречий и возможностью удалить данные по запросу.

Обязательные критерии

Выполненное задание должно удовлетворять 6 обязательным критериям.

№	Критерий	Занятие
1	Реализован диалоговый контур агента на LangGraph: состояние диалога с накоплением слотов через structured output, кратковременная память в рамках сессии через checkpointer и долговременная память между сессиями через Store, а ответ агента обязан ссылаться на конкретный найденный источник.	6

№	Критерий	Занятие
2	Реализовано управление контекстом под заданный бюджет: подсчёт расхода токенов по фактическим данным ответа модели, суммаризация выпадающей из окна истории специализированным под вашу задачу промптом.	6
3	Реализован гибридный поиск по базе знаний: лексический поиск и поиск на эмбедингах, слияние двух выдач и переупорядочивание кандидатов, а сам поиск оформлен как инструмент, который агент вызывает сам, а не как фиксированная цепочка.	7
4	Реализован отбор документов по метаданным с учётом актуальности редакции на нужный момент времени, и показано, что один и тот же вопрос при разных значениях этого параметра даёт разные корректные ответы; хранилища разведены по ролям памяти с осмысленным сроком жизни записей.	7
5	Спроектирован слой памяти о пользователе: явная схема записи, разные горизонты хранения для разных видов сведений.	8
6	Реализованы целостность и управляемость памяти: журнал аудита операций над записями, удаление данных по запросу пользователя с сохранением тех сведений, которые обязаны храниться, поиск по памяти с учётом не только смысловой близости, но и свежести записи, а также перенос дорогой переработки памяти за пределы диалога.	8

Формат сдачи

- Репозиторий с кодом и подробным README (возможности агента, запуск, способ проверки).
- Воспроизводимый запуск одной командой: `venv + скрипт запуска (run.sh / make run)` или Docker – на ваш выбор.
- Подготовленная база знаний с документами и их редакциями.
- скринкаст до 10 минут с демонстрацией работы вашего решения.

Задание 3. Управляемый ИИ-агент

Необходимо подготовить индивидуальный итоговый проект: работающего управляемого ИИ-агента, который умеет получать факты через инструменты, выбирать следующий шаг по результатам наблюдений, делегировать узкую задачу дочернему агенту и безопасно доводить подтверждённое действие до выполнения. Проект должен показывать, что языковая модель лишь предлагает следующий шаг, а права на вызов функций, переходы между состояниями, лимиты, повторы и внешние эффекты контролирует обычный Python-код.

Вы можете развить сквозной учебный сценарий с двойным списанием по платежу Р-77 или выбрать собственную предметную область. Важна не тема, а целостный и проверяемый путь от запроса до результата.

Ожидаемый результат: на вход система получает запрос пользователя, далее она: определяет, можно ли ответить без модели, нужно ли уточнение или следует запустить агентный маршрут; передает GigaChat схемы доступных инструментов; проверяет каждое предложение модели до вызова Python-функции; обновляет явное состояние и выбирает маршрут по проверенным наблюдениям; при необходимости подключает предметный навык и передает узкое поручение дочернему агенту; превращает внешний эффект в предложение, требующее точного подтверждения; после подтверждения передает задачу в надежный контур и возвращает понятный итог вместе с трассировкой.

Обязательные критерии

Для зачёта проект должен удовлетворять всем шести обязательным критериям.

№	Критерий	Что нужно показать	Занятие
1	GigaChat и Function Calling	Агент использует GigaChat и минимум два прикладных инструмента. Минимум один инструмент читает актуальные данные. Исходный tool_call_id сохраняется в ответном ToolMessage.	1
2	Программная граница доверия	До исполнения Python-код проверяет имя инструмента, закрытую схему и типы аргументов, разрешение и текущее состояние. Внешний эффект не выполняется без точного подтверждения. Число шагов и обращений к модели ограничено кодом.	1–2
3	Ветвящийся и устойчивый ReAct-цикл	В системе есть явное состояние и минимум два маршрута, один из которых выбирается по наблюдению от инструмента. Недопустимый переход блокируется. Для безопасного чтения реализованы ограниченные повторы и кэш только успешных результатов; после исчерпания попыток агент завершается с понятной деградацией.	2
4	Навык и дочерний агент	Минимум одна предметная инструкция оформлена как версионизируемый навык. Управляющий агент делегирует минимум одну задачу дочернему агенту. В поручении передаются только нужные факты, ограниченный набор инструментов и отдельный бюджет. Родитель проверяет автора, статус, факты и источник результата. Можно использовать Deep Agents или явно реализованный AgentHarness с теми же границами.	3

№	Критерий	Что нужно показать	Занятие
5	Наблюдаемость	Контур записывает безопасную трассировку: маршрут, смену состояний, вызовы инструментов и дочернего агента, повторы и причину остановки. Выводятся как минимум две метрики, например число обращений к модели и число вызовов инструментов. Секреты и чувствительные данные в события не попадают.	2–3
6	Надёжное выполнение подтверждённой задачи	Реализован порядок publish, reserve, сохранить результат, ack. Есть приоритет, ограниченные повторы и DLQ, идемпотентная бизнес-операция. message_id устраняет повтор одного сообщения, а idempotency_key повтор одной бизнес-команды.	4

Обязательные сценарии демонстрации

В коде, тестах или скринкасте необходимо показать пять групп сценариев:

1. **Успешный длинный маршрут.** Агент делает несколько зависимых чтений, выбирает ветку по наблюдению, подключает навык и дочернего агента, после чего возвращает ответ с проверенными фактами.
2. **Короткий или дешёвый маршрут.** Точное правило обходит модель либо обычное наблюдение завершает цикл раньше. В трассировке видна экономия обращений к GigaChat.
3. **Отклонённое предложение.** Неизвестный инструмент, неверные аргументы, недопустимый переход или неподтверждённое действие не вызывают обработчик. Запуск завершается с понятной причиной.
4. **Временная ошибка.** Безопасное чтение повторяется строго в пределах лимита. Затем система либо получает успех, либо переходит в деградацию; успешное повторное чтение берётся из кэша.
5. **Повторная доставка после сбоя.** После сохранения результата, но до ack, имитируется сбой. Повторная выдача завершает задачу, но не повторяет бизнес-операцию. Отдельно показывается перегрузка или исчерпание попыток с переводом в DLQ.

Формат сдачи

- README.md: задача, архитектура агента, команды запуска и как проверить обязательные сценарии;
- файл зависимостей (pyproject.toml, requirements.txt или эквивалент) и .env.example с именами переменных без значений;
- минимум один SKILL.md и справочные материалы навыка;
- воспроизводимый скрипт или автотесты, прогоняющие все обязательные сценарии одной командой;

- скринкаст до 10 минут: разбор архитектуры и демонстрация успешных и аварийных маршрутов.

Дополнительная часть. Оценка ИИ-агентов

В рамках проекта вы можете дополнить уже разработанного ИИ-агента полноценным контуром оценки качества. В качестве основы можно использовать агента, созданного в предыдущих заданиях, либо собственного агента, разработанного вне рамок курса. В проекте необходимо продемонстрировать владение продвинутыми методами построения LLM-as-a-Judge: рубрикацией, декомпозицией судей, использованием нескольких судей, оценкой неопределённости и другими рассмотренными методами. Основная задача: реализовать поверх существующего агента две взаимодополняющие системы оценки:

- оффлайн-оценку для регрессионного тестирования новых версий агента;
- онлайн-оценку для оценки качества конкретного запуска агента непосредственно во время его работы.

Оффлайн-оценка должна запускать выбранную конфигурацию агента на заранее подготовленном наборе задач (тестовой корзине) и рассчитывать одну или несколько целевых метрик качества (занятие 9).

Онлайн-оценка должна быть встроена непосредственно в архитектуру агента и формировать оценку качества текущего запуска в виде дополнительного артефакта его работы. В LangGraph-проекте это, как правило, означает добавление одного или нескольких новых узлов после выполнения основной агентной логики.

Для обеих систем можно использовать любые метрики качества агентов, рассмотренные на занятиях, либо их обоснованные производные: успешность выполнения задачи, корректность вызова инструментов, качество поиска и памяти и так далее.

Ожидаемый результат

Должны быть реализованы два связанных модуля:

Модуль оффлайн-оценки представляет собой отдельный Python-скрипт или приложение, при необходимости упакованное в Docker. Модуль получает конфигурацию агента через аргументы командной строки, YAML-файл или другой явно определённый интерфейс, запускает агента на заранее подготовленной тестовой корзине и рассчитывает итоговые метрики качества.

Результатом его работы должен быть машиночитаемый отчёт (например YAML-файл).

Модуль онлайн-оценки должен быть встроен непосредственно в существующего агента. После завершения основной работы агент запускает дополнительный оценочный контур, а полученный результат сохраняется в его состоянии. Таким образом, необходимо показать явное изменение архитектуры агента: исходная версия возвращала только результат выполнения задачи, а новая версия дополнительно возвращает оценку качества конкретного запуска.

Обязательные критерии

Каждый из перечисленных ниже критериев должен быть реализован либо в оффлайн-контуре, либо в онлайн-контуре оценки (если не прописано отдельно). При желании одну и ту же методику можно использовать в обоих контурах, однако это не требуется.

№	Критерий	Что нужно показать
1	LLM-as-a-Judge	Основные метрики качества агента рассчитываются при помощи LLM-судьи. Судья возвращает результат в строго определённом формате: число, категорию из множества с низкой кардинальностью либо результат попарного сравнения двух версий агента (занятия 9 и 11).
2	Рубрикация	Оценка разбита на отдельные рубрики, явно описанные в промпте LLM-судьи (занятие 11). Каждая рубрика имеет низкую кардинальность, предпочтительно бинарную или тернарную шкалу. Итоговая агрегированная оценка рассчитывается программно после получения результатов по отдельным рубрикам.
3	Декомпозиция LLM-судей	Для разных рубрик вызываются независимые LLM-судьи (занятие 12). Каждый судья получает промпт только для своей рубрики и не зависит от результатов других судей. Итоговая оценка агрегируется программно.
4	LLM-жюри	Для минимум одной рубрики используется несколько независимых судей (не менее 3). Результаты объединяются подходящим способом: например majority vote для категориальных вердиктов или усреднением для числовой оценки (занятие 12). Желательно использовать разные модели; если доступна только одна (например GigaChat), разные конфигурации судей можно имитировать различными параметрами генерации.
5	Неопределённость LLM-судьи	Вместе с вердиктом LLM-судьи рассчитывается оценка неопределённости доступным для выбранного провайдера способом (занятие 12). Если оценка неопределённости используется в онлайн-контуре, при превышении заданного порога должен запускаться Human-in-the-Loop (либо его программная заглушка).

№	Критерий	Что нужно показать
6	Анализ краевых случаев	В тестовую корзину оффлайн-оценки отдельно включаются edge cases: сценарии, не относящиеся к основному рабочему потоку, но способные существенно нарушить работу агента (занятие 12). Для них рассчитываются отдельные метрики, которые отдельно выделяются в итоговом отчёте.

Что можно сделать сверх обязательного

№	Модуль	Что нужно показать
1	Адаптивные рубрики	В онлайн-контуре реализуется адаптивная рубрикация: набор критериев для конкретной задачи определяется LLM динамически (занятие 12). После завершения основной работы агента, но до вызова оценочных судей, добавляется отдельный модуль генерации рубрик. Полученные рубрики затем используются для оценки конкретного запуска.
2	Калибровка и оценка LLM-судьи	Подготавливается небольшой эталонный датасет, размеченный разработчиком или экспертом. Согласованность LLM-судьи с экспертной разметкой измеряется формальной метрикой, например Cohen's kappa (занятия 10 и 11). Далее при помощи DSPy выполняется автоматическая оптимизация промпта судьи с целью максимизации выбранной метрики согласованности (занятие 12). Полученный промпт используется в основном пайплайне и хранится в проекте как отдельный файл.

Обязательные сценарии демонстрации

В коде, тестах или скринкасте необходимо продемонстрировать как оффлайн-, так и онлайн-контур оценки.

- Оффлайн-регрессионное тестирование.** Показать запуск отдельного Python-модуля или Docker-контейнера, который получает конфигурацию агента, запускает его на подготовленной корзине задач и автоматически рассчитывает метрики качества.
- Отчёт оффлайн-оценки.** После завершения тестирования формируется YAML-файл или другой машиночитаемый отчёт с основными метриками агента. Если реализован анализ edge cases, результаты для них выносятся отдельно.
- Интеграция онлайн-оценки.** Показать изменения в архитектуре агента: добавление одного или нескольких узлов для LLM-судьи, оценки неопределённости, адаптивной рубрикации или других выбранных компонентов.

4. **Онлайн-оценка конкретного запуска.** Выполнить запрос к агенту и показать, что после завершения работы его состояние содержит новые поля с результатами оценки конкретного запуска.
5. **Сравнение состояния до и после доработки.** Явно показать инкрементальное изменение агента: какие артефакты возвращала исходная версия и какие дополнительные данные появились после внедрения онлайн-оценки.

Пример ожидаемого изменения:

До внедрения оценки:

```
{  
  "answer": "...",  
  "messages": [...]  
}
```

После внедрения оценки:

```
{  
  "answer": "...",  
  "messages": [...],  
  "online_evaluation": {  
    "rubrics": {...},  
    "score": 0.84,  
    "uncertainty": 0.09,  
    "decision": "accept"  
  }  
}
```